케이녹 C언어 7회차 과제

17기 최동현 (202511688)

1. 파일 입출력, 포인터 (File I/O, FILE)

• 파일 입출력(File Input/Output)이란?

: C언어에서 **파일 입출력(File I/O)**은 운영체제(OS)의 **파일 디스크립터(File Descriptor)**와 연결된 **스트림(Stream)**을 통해 이루어지는 방식입니다.

FILE은 이 스트림을 제어하기 위해 필요한 버퍼의 주소, 현재 파일 오프셋, 상태 플래그 등을 담고 있는 **C 표준 라이브러리(stdio.h)**의 **구조체**입니다. (ex. typedef struct { 버퍼주소; 파일오프셋; 상태플래그; ... } FILE;)

```c
// 파일 포인터(File Pointer)를 사용한 파일 열기(fopen)
FILE *포인터명 = fopen("파일경로", "모드");
// 모드: "r"(read), "w"(write), "a"(Append)
// + 텍스트 파일이 아닌 파일을 열때는 바이너리인 b를 붙여야 데이터 왜곡이 없음.
FILE *fp = fopen("Kknock.png","rb");

// 파일 닫기(fclose)
int 변수이름 = fclose(포인터명); // 반환 결과 반환
// 반환 값으로 0(성공) 혹은 EOF(-1|실패)이 반환 됨
fclose(fp);
```

1-1. 파일 입출력, 포인터 (File I/O, FILE)

- **fopen()과 fclose()** :

1) fopen: 운영체제에 파일 **접근**을 **요청**하고, 성공 시 커널 공간의 파일을 제어할 수 있는 **FILE 구조체**를 메모리에 **동적 할당**하여 그 **포인터**를 **반환**합니다.

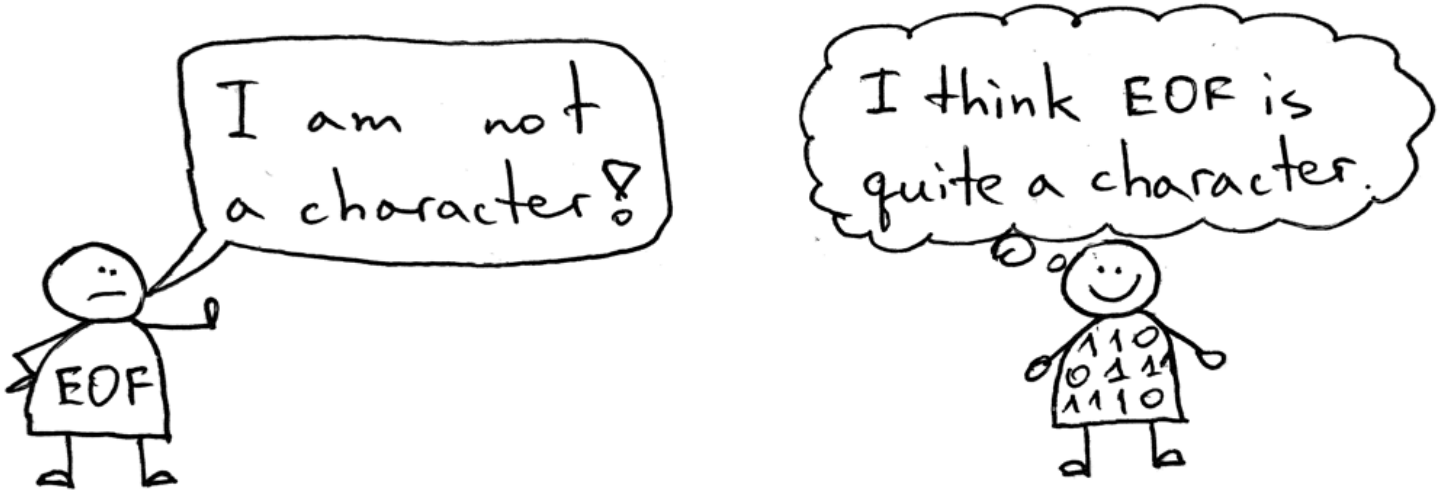
2) fclose: 파일 스트림 버퍼에 남아있는 데이터를 디스크로 밀어내고(기록), 운영체제에 할당받은 **파일 디스크립터**와 **FILE 구조체 메모리**를 **반환(free)**합니다.

open (파일 열기) → *파일 읽기 / 쓰기* → *close (파일 닫기)*

1-2. 파일 입출력, 포인터 (File I/O, FILE)

- **EOF(End of File)이란?**

: C언어에서 **EOF**는 파일의 끝을 나타내는 제어 문자가 아닌, 더 **이상 읽을 데이터**가 **없거나 시스템 에러**가 발생했음을 호출자에게 알리기 위해 반환하는 **매크로 상수(정수)**입니다. (통상적으로 -1로 정의 됨)



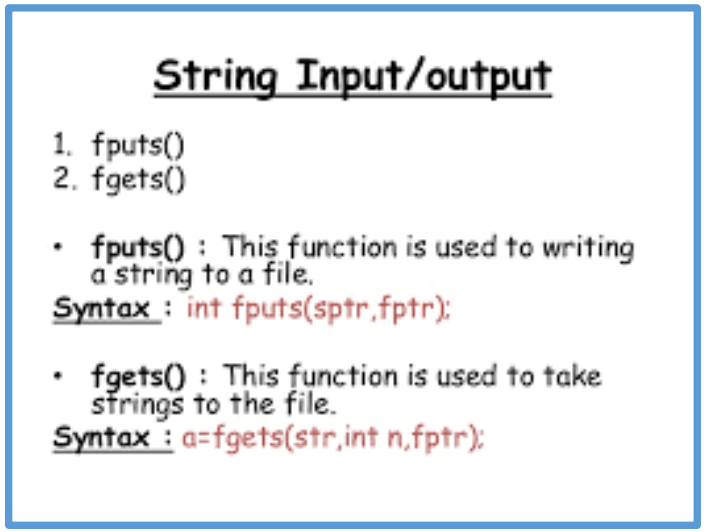
1-3. 파일 입출력, 포인터 (File I/O, FILE)

- **텍스트 입출력 함수 [fgetc(), fgets(), fputc(), fputs()]** :

  - **1) fgetc / fgets** : 스트림에서 **1바이트(문자 1개)씩** 메모리로 가져오거나 파일로 전송합니다.

  - **2) fgets** : fgets(buf, size, fp); 로 사용하며, **스트림(stdin, fp, ...)**에서 데이터를 읽어 지정된 **버퍼**에 **저장**합니다. **개행 문자(₩n)**를 만나거나 **(size – 1) 바이트**를 읽을 때까지 수행됩니다.

  - **3) fputs :** fputs(str, fp); 로 사용하며, 메모리의 **문자열**(널 바이트 이전까지의 데이터)을 **파일 스트림**으로 **전송**합니다. (개행 문자를 자동으로 추가하지 않음)



1-3. 파일 입출력, 포인터 (File I/O, FILE)

- C언어 텍스트 입출력 함수 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>

int main(void) {
    FILE *fp = NULL;
    char buf[100];

    fp = fopen(filename: "test.txt", mode: "w");
    if (fp == NULL) { printf("파일 열기 실패!\n"); return 1; }

    printf("문자열 입력: ");
    fgets(buf, size: sizeof(buf), stdin);
    fputs(buf, fp);
    fclose(fp);

    fp = fopen(filename: "test.txt", mode: "r");
    if (fp == NULL) { printf("파일 열기 실패!\n"); return 1; }

    printf("\n=== 파일 내용 ===\n");
    if (fgets(buf, size: sizeof(buf), fp) != NULL) {
        printf("내용: ");
        fputs(buf, stdout);
    }

    return 0;
}
```

```
~/Desktop/CHOI/KKnock/Day7  vim main.c

문자열 입력: K.knock

=== 파일 내용 ===
내용: K.knock

계속하려면 엔터 혹은 명령을 입력하십시오

~/Desktop/CHOI/KKnock/Day7  cat ./test.txt
K.knock
```

1-4. 파일 입출력, 포인터 (File I/O, FILE)

- **포맷 지정 입출력 함수 [fscanf(), fprintf()]**

: 메모리에 저장된 정수나 실수 등의 **이진데이터(Binary)**를 사람이 읽을 수 있는 **아스키(ASCII) 텍스트**로 **변환**하여 **파일**에 쓰거나, 그 **반대로** **파일**의 **텍스트**를 **파싱**하여 변수의 **메모리**에 **저장**하는 함수입니다.

```
// 포맷 파싱 및 변수 저장
fscanf(파일포인터, "포맷문자열", &변수명);
fscanf(fp, "%d", &a); // 파일에서 10이 있었다면 정수형으로 포맷 후 a변수에 넣음

// 포맷 변환 및 파일 출력
fprintf(파일포인터, "포맷문자열", 값);
fprintf(fp, "%d", a) // a에 있는 값이 10이라면 정수형 포맷 문자열 10을 fp에 넣음
```

1-5. 파일 입출력, 포인터 (File I/O, FILE)

- **바이너리 블록 입출력 [fread(), fwrite()]**

: **구조체 배열**이나 **멀티미디어 데이터**(*이미지*, *실행파일* 등)를 텍스트 변환 과정 없이, 메모리에 적재된 **비트 형태** 그대로 파일에 **전송**하거나 **읽어오는** 함수입니다. 데이터의 **크기(size)**와 항목 **개수(count)**를 인자로 받아 정확한 **바이트 블록 단위**의 메모리 복사 연산을 수행합니다.

```
// 바이너리 데이터 읽기
fread(저장할버퍼포인터, 단일크기, 항목개수, 파일포인터);

// 바이너리 데이터 쓰기
fwrite(출력할데이터포인터, 단일크기, 항목개수, 파일포인터);

// 예시
typedef struct { int id; char pw[20]; } Users;
Users udata[2] = { {2025,"asdf"}, {2026,"fdsa"} };
FILE* fp = fopen("kknock.dat","wb"); // 이진파일이라 b(Binary) 추가
fwrite(&udata, sizeof(Users), 2, fp); // fp에 구조체 배열 내용 2개 추가
```

1-6. 파일 입출력, 포인터 (File I/O, FILE)

- **파일 스트림 포인터 제어 [ftell(), rewind(), fseek()]**

  : **파일 스트림**은 내부적으로 현재 **읽거나 쓰고 있는 위치**를 나타내는 **파일 위치 지시자**(File Position Indicator / Offset)를 가집니다.

  1) ftell : 파일의 **시작점**으로부터 현재 **지시자가 위치한 곳**까지의 **바이트 수(Offset)**를 **반환**합니다.

  2) rewind : **파일 위치 지시자**를 파일의 **맨 처음(Offset 0)**으로 **강제 초기화**합니다.

  3) fseek : 지시자의 위치를 기준점(**SEEK_SET**: 시작, **SEEK_CUR**: 현재, **SEEK_END**: 끝)으로부터 **지정된 오프셋만큼** 앞뒤로 **강제 이동**시킵니다.

1-6. 파일 입출력, 포인터 (File I/O, FILE)

- C언어 파일 스트림 포인터 제어 (예시 코드화면 및 실행화면)

```c
#include <stdio.h>

int main(void) {
    FILE *fp = NULL;
    fp = fopen(filename: "test.dat", mode: "rb");

    if (fp != NULL) {
        fseek(fp, 0, SEEK_END); // 파일 위치 지시자를 맨뒤로 보냄
        int fileSize = ftell(fp);
        // 시작점 부터 현재 위치(SEEK_END)까지 바이트 수 반환
        printf("파일의 전체 크기: %d 바이트\n", fileSize);
        rewind(fp); // 커서를 다시 맨 앞으로 초기화 시킴
        fclose(fp);
    }

    return 0;
}
```

```
~/Desktop/CHOI/KKnock/Day7  head -c 1024 /dev/urandom > test.dat

~/Desktop/CHOI/KKnock/Day7  vim main.c

파일의 전체 크기: 1024 바이트

계속하려면 엔터 혹은 명령을 입력하십시오
```

2. 실습 과제 (백준 성지키기)

문제

영식이는 직사각형 모양의 성을 가지고 있다. 성의 1층은 몇 명의 경비원에 의해서 보호되고 있다. 영식이는 모든 행과 모든 열에 한 명 이상의 경비원이 있으면 좋겠다고 생각했다.

성의 크기와 경비원이 어디있는지 주어졌을 때, 몇 명의 경비원을 최소로 추가해야 영식이를 만족시키는지 구하는 프로그램을 작성하시오.

입력

첫째 줄에 성의 세로 크기 N과 가로 크기 M이 주어진다. N과 M은 50보다 작거나 같은 자연수이다. 둘째 줄부터 N개의 줄에는 성의 상태가 주어진다. 성의 상태는 .은 빈칸, X는 경비원이 있는 칸이다.

출력

첫째 줄에 추가해야 하는 경비원의 최솟값을 출력한다.

예제 입력 1 복사

```
4 4
....
....
....
....
```

예제 출력 1 복사

```
4
```

2. 실습 과제 (백준 성지키기)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 0, m = 0;
    scanf("%d %d", &n, &m);

    char **castle = (char**)malloc(size: sizeof(char*) * n);
    for(int i = 0; i < n; i++) {
        *(castle + i) = (char*)malloc(size: sizeof(char) * m);
    }
    int rowGuard[50] = {0}; // 행에 경비원 있는 곳표시용
    int colGuard[50] = {0}; // 열에 경비원 있는 곳표시용
    int hasGuard = 0; // 경비원 있는지 확인 bool용
    int rowNeed = 0, colNeed = 0; // 행과 열에 필요한 경비원 명수 표기용

    for(int i = 0; i < n; i++) {
        scanf("%s", castle[i]);
    }

    for(int i = 0; i < n; i++) {
        hasGuard = 0;
        for(int j = 0; j < m; j++) {
            if (castle[i][j] == 'X') {
                hasGuard = 1; // 경비원 표시
                break; // 표시용이라 1만 찍고 나가기
            }
        }
        rowGuard[i] = hasGuard;
    }
    for(int i = 0; i < m; i++) {
        hasGuard = 0;
        for(int j = 0; j < n; j++) {
            if (castle[j][i] == 'X') {
                hasGuard = 1; // 경비원 표시
                break; // 표시용이라 1만 찍고 나가기
            }
        }
        colGuard[i] = hasGuard;
    }

    for(int i = 0; i < n; i++) {
        if (rowGuard[i] != 1) {
            rowNeed++; // 행 경비원이 없는 곳 만큼 추가
        }
    }
    for(int i = 0; i < m; i++) {
        if (colGuard[i] != 1) {
            colNeed++; // 열 경비원 없는 곳 만큼 추가
        }
    }
    printf("%d\n", rowNeed > colNeed ? rowNeed : colNeed);

    return 0;
}
```

NORMAL ./KKnock/Day7/defense

```
~/Desktop/CHOI  vim ./KKnock/Day7/defenseCastle.c

3 5
XX...
.XX..
...XX
0

계속하려면 엔터 혹은 명령을 입력하십시오
```

2. 실습 과제 (학생 성적 관리 프로그램)

문제1: 학생 성적 관리 프로그램

구조체를 활용하여 데이터를 체계적으로 관리하는 프로그램을 작성합니다.

요구사항

1. 다음과 같은 학생 구조체를 정의하세요:
   - 이름 (`char name[50]`)
   - 학번 (`int id`)
   - 국어, 영어, 수학 점수 (`int korean, english, math`)
   - 평균 (`float average`)
2. 다음 기능을 구현하세요.
   - 각 학생의 정보를 입력받고, 평균을 계산하세요.
   - 전체 학생의 정보를 표 형식으로 출력하세요.
   - 평균이 가장 높은 학생을 출력하세요.

> 최대 학생 수는 3명입니다. 3번 입력 받으면 됩니다.

제약 조건

- 구조체 배열을 사용하세요(동적 할당은 선택).
- 구조체 포인터를 활용하는 함수를 최소 1개 포함하세요.

예시 실행 결과

```
=== 학생 성적 입력 ===

[1번째 학생]
이름: 김철수
학번: 202401
국어, 영어, 수학 점수: 85 90 78

[2번째 학생]
이름: 이영희
학번: 202402
국어, 영어, 수학 점수: 92 88 95

[3번째 학생]
이름: 박민수
학번: 202403
국어, 영어, 수학 점수: 70 82 80


이름        학번        국어  영어  수학  평균
------------------------------------------------------------
김철수      202401      85    90    78    84.33
이영희      202402      92    88    95    91.67
박민수      202403      70    82    80    77.33
------------------------------------------------------------

★ 성적 우수자 ★
이름: 이영희 (평균: 91.67점)
```

2. 실습 과제 (학생 성적 관리 프로그램)

```c
#include <stdio.h>
#include <string.h>

void getLine(int n) {
    for(int i = 0; i < n; i++) {
        printf("-");
    }
    printf("\n");
}

typedef struct student {
    char name[50];
    int id;
    int korean, english, math;
    float average;
} Student;
```

1

```c
int main(void) {
    Student stdList[3];
    char buf[50];
    int nameLength = 0;
    float maxAver = 0.0;

    printf("=== 학생 성적 입력 ===\n");
    for(int i = 0; i < 3; i++) {
        printf("\n[%d번째 학생]\n", i + 1);
        printf("이름 : ");
        fgets(stdList[i].name, size: sizeof(stdList[i].name), stdin);
        nameLength = strlen(s: stdList[i].name);
        if (stdList[i].name[nameLength - 1] ==  '\n') stdList[i].name[nameLength - 1] = '\0';
        printf("학번 : ");
        fgets(buf, size: sizeof(buf), stdin);
        sscanf(buf, "%d", &stdList[i].id);
        printf("국어, 영어, 수학 점수 : ");
        fgets(buf, size: sizeof(buf), stdin);
        sscanf(buf, "%d %d %d", &stdList[i].korean, &stdList[i].english, &stdList[i].math);
        stdList[i].average = (float)(stdList[i].korean + stdList[i].english + stdList[i].math) / 3.0;
    }

    maxAver = (stdList[0].average > stdList[1].average) ?
        (stdList[0].average > stdList[2].average ? stdList[0].average : stdList[2].average) :
        (stdList[1].average > stdList[2].average ? stdList[1].average : stdList[2].average);

    //printf("이름\t학번\t국어\t영어\t수학\t평균\n");
    printf("\n%-10s %-10s %-5s %-5s %-5s %-7s\n", "이름", "학번", "국어", "영어", "수학", "평균");
    getLine(n: 60);
    for(int i = 0; i < 3; i++) {
    printf("%-10s %-10d %-5d %-5d %-5d %-7.2lf\n",
                stdList[i].name, stdList[i].id,
                stdList[i].korean, stdList[i].english, stdList[i].math, stdList[i].average);
    }
    getLine(n: 60);
```

2

```c
    printf("* 성적 우수자 *\n");
    for(int i = 0; i < 3; i++) {
        if (maxAver == stdList[i].average) {
            printf("이름 : %s (평균 : %.2lf점)\n", stdList[i].name, maxAver);
        }
    }

    return 0;
}
```

3

2. 실습 과제 (학생 성적 관리 프로그램)

```
100%                                                    8%
~/Desktop/CHOI   vim ./KKnock/Day7/gradeManage.c

=== 학생 성적 입력 ===

[1번째 학생]
이름: 김귀룡
학번: 202601
국어, 영어, 수학 점수: 85 90 78

[2번째 학생]
이름: 이귀룡
학번: 202602
국어, 영어, 수학 점수: 92 88 95

[3번째 학생]
이름: 박귀룡
학번: 202603
국어, 영어, 수학 점수: 70 82 80

이름      학번      국어 영어 수학 평균
------------------------------------------------------------
김귀룡   202601     85    90    78    84.33
이귀룡   202602     92    88    95    91.67
박귀룡   202603     70    82    80    77.33
------------------------------------------------------------
* 성적 우수자 *
이름: 이귀룡 (평균: 91.67점)
```

2. 실습 과제 (동적 연락처 관리 프로그램)

문제2: 동적 연락처 관리 프로그램

동적 할당과 구조체를 결합하여 실용적인 데이터 관리 프로그램을 구현합니다.

요구사항

`/* 빈칸 */` 채우기

- 다음 기능을 구현하세요.
  - `initManager` 함수에서 구조체를 동적으로 할당하는 코드를 작성해주세요.
  - `addContact` 함수에서 초기 용량을 초과했을 때, 구조체를 재할당하는 코드를 작성해주세요.
  - `deleteContact` 함수에서 내부 멤버의 메모리를 해제하는 코드를 작성해주세요.
  - `freeManager` 함수에서 모든 메모리를 해제하는 코드를 작성해주세요.

제약 조건

- 이름과 이메일은 동적 할당으로 저장하세요.
- 배열 용량 초과 시 `realloc` 으로 확장하세요. (초기 용량: 5)
- 삭제 시 해당 연락처의 동적 메모리를 해제하세요.
- `enum` 을 활용하여 그룹을 관리하세요.
- 프로그램 종료 시, 모든 메모리를 해제해야 합니다.

예시 실행 결과

```
--- 연락처 관리 시스템 (용량: 0/5) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 1
이름: 홍길동
전화번호: 01099999999
이메일:  load@gmail.com
그룹 (0:FAMILY, 1:FRIEND, 2:WORK): 1
연락처가 저장되었습니다.

--- 연락처 관리 시스템 (용량: 1/5) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 1
이름: adams
...

--- 연락처 관리 시스템 (용량: 5/5) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 1
>> 용량이 10로 확장되었습니다.
이름: admin
전화번호: 0
이메일: admin@gmail.com
그룹 (0:FAMILY, 1:FRIEND, 2:WORK): 2
연락처가 저장되었습니다.
```

2. 실습 과제 (동적 연락처 관리 프로그램)

```c
// 그룹 정의
typedef enum { FAMILY, FRIEND, WORK } Group;
const char* GROUP_NAMES[] = { "FAMILY", "FRIEND", "WORK" };

// 연락처 구조체
typedef struct {
    char *name;         // 동적 할당
    char phone[15];
    char *email;        // 동적 할당
    Group group;
} Contact;

// 관리 시스템 구조체
typedef struct {
    Contact *list;
    int count;
    int capacity;
} ContactManager;
```

프로그램 열거형 및 구조체

2. 실습 과제 (동적 연락처 관리 프로그램)

```
// 구조체를 동적으로 할당하는 코드를 작성해주세요.
m->list = (Contact*)malloc(sizeof(Contact) * m->capacity);
```

```
// capacity 만큼 구조체를 재할당 해주세요.
Contact *tmp = (Contact*)realloc(m->list, sizeof(Contact) * m->capacity);
```

```
// 내부 멤버 메모리를 해제하는 코드를 작성해주세요.
free(m->list[i].name);
free(m->list[i].email);
```

```
// 모든 메모리 해제하는 코드를 작성해주세요.
for (int i = 0; i < m->count; i++) {
    free(m->list[i].name);
    free(m->list[i].email);
}
free(m->list);
```

2. 실습 과제 (동적 연락처 관리 프로그램)

```
--- 연락처 관리 시스템 (용량: 3/5) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 1
이름: choi
전화번호: 4
이메일: choi@mail.net
그룹 (0:FAMILY, 1:FRIEND, 2:WORK): 0
연락처가 저장되었습니다.

--- 연락처 관리 시스템 (용량: 4/5) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 1
이름: jung
전화번호: 5
이메일: jung@mail.net
그룹 (0:FAMILY, 1:FRIEND, 2:WORK): 1
연락처가 저장되었습니다.

--- 연락처 관리 시스템 (용량: 5/5) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 1
>> 용량이 10로 확장되었습니다.
이름: admin
전화번호: 0
이메일: admin@mail.net
그룹 (0:FAMILY, 1:FRIEND, 2:WORK): 2
연락처가 저장되었습니다.
```

```
--- 연락처 관리 시스템 (용량: 6/10) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 2
검색할 이름: admin
[WORK] admin | 0 | admin@mail.net

--- 연락처 관리 시스템 (용량: 6/10) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 3
삭제할 이름: admin
삭제 완료되었습니다.

--- 연락처 관리 시스템 (용량: 5/10) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 4

=== 전체 목록 ===
1. [FAMILY] kim | 1 | kim@mail.net
2. [FRIEND] lee | 2 | lee@mail.net
3. [WORK] park | 3 | park@mail.net
4. [FAMILY] choi | 4 | choi@mail.net
5. [FRIEND] jung | 5 | jung@mail.net

--- 연락처 관리 시스템 (용량: 5/10) ---
1. 추가  2. 검색  3. 삭제  4. 전체출력  5. 그룹필터  6. 종료
선택: 5
출력할 그룹 (0:FAMILY, 1:FRIEND, 2:WORK): 0

=== 그룹 필터 결과 ===
[FAMILY] kim | 1 | kim@mail.net
[FAMILY] choi | 4 | choi@mail.net
```

감사합니다

17기 최동현